



MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1 2025/2026

WEEK 7: INTERFACING A PLC WITH A MICROCONTROLLER AND A PC OVER ETHERNET/IP

SECTION 2

GROUP 16

LECTURER: ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO

Date of Experiment: Monday, 17 November 2025

Date of Submission: Monday, 24 November 2025

NO.	GROUP MEMBERS	MATRIC NO
1.	MUHAMMAD HAIKAL ZULHILMI BIN FAIROF	2313025
2.	MUHAMMAD FAIZ IZWAN BIN NOR EFFENDI	2313509
3.	AINUL JAARIAH BINTI ALIUDIN	2312664

ABSTRACT

This experiment looks at how OpenPLC Editor can be used to design, simulate, and implement ladder logic on an Arduino microcontroller over Ethernet/IP. A blinking LED program and a Start-Stop motor control circuit were created using ladder diagrams. The goal was to understand PLC-microcontroller interfacing, variable addressing, and program uploading using OpenPLC Runtime. The system was tested with Arduino's digital I/O pins, push buttons, and LEDs. The results show that OpenPLC is a reliable platform for teaching PLC programming concepts that does not require industrial PLC hardware. The experiment successfully demonstrated signal control, program transfer, and functional verification of ladder logic on physical hardware.

TABLES OF CONTENTS

ABSTRACT.....	2
TABLES OF CONTENTS.....	3
1.0 INTRODUCTION.....	4
2.0 MATERIAL AND EQUIPMENT.....	5
3.0 EXPERIMENTAL SETUP.....	6
4.0 METHODOLOGY.....	7
4.1 Software Preparation.....	7
4.2 Ladder Diagram Creation.....	7
4.3 Physical Address Mapping.....	7
4.4 Uploading Ladder Logic to the Arduino.....	7
4.5 Hardware Setup.....	8
4.6 Testing and Verification.....	8
5.0 DATA COLLECTION.....	9
6.0 DATA ANALYSIS.....	10
6.1 LED Blinking Circuit (Timer-Based Analysis).....	10
6.2 Start–Stop Control Circuit (Latching Logic Analysis).....	10
7.0 RESULTS.....	11
8.0 DISCUSSIONS.....	12
9.0 CONCLUSION.....	13
10.0 RECOMMENDATION.....	14
REFERENCES.....	15
APPENDICES.....	16
Appendix A.....	16
ACKNOWLEDGEMENT.....	18
STUDENTS DECLARATION.....	18
<i>Certificate of Originality and Authenticity.....</i>	<i>19</i>

1.0 INTRODUCTION

Programmable Logic Controllers (PLCs) are critical industrial automation devices that control electromechanical processes in assembly lines, machines, and robotic systems. As modern automation incorporates Industry 4.0 principles, PLCs are increasingly working alongside microcontrollers and personal computers via standardized communication protocols like Ethernet/IP.

In this experiment, the OpenPLC a fully open-source PLC development environment was used to demonstrate PLC programming concepts using ladder diagrams. The software enables users to design logic circuits, simulate operations, and deploy them to low-cost hardware platforms like Arduino.

The lab session focuses on developing two PLC applications.

1. A simple blinking LED with contact and coil elements.
2. A Start-Stop control circuit based on push buttons and an output LED.

This activity gives students hands-on experience with creating ladder logic, managing variable associations, configuring physical addressing, transferring programs to the Arduino, and verifying real-world functionality.

2.0 MATERIAL AND EQUIPMENT

- OpenPLC Editor software
- Arduino Board
- 2 Push Button Switches
- LED
- Resistors
- Jumper Wires
- Breadboard

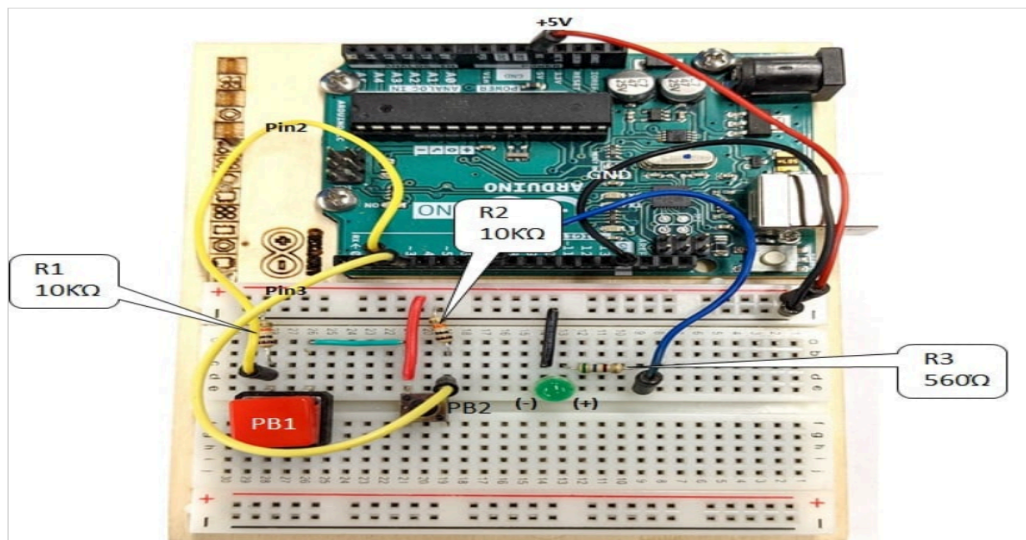


Figure 4. The Arduino Uno PLC electronic circuit schematic diagram.



Fig.1: OpenPLC Editor website

3.0 EXPERIMENTAL SETUP

The experimental setup consists of three main components:

1. **OpenPLC Editor (PC)** – used to design ladder logic, assign variables, map I/O addresses, and transfer PLC programs to the Arduino.
2. **Arduino Board** – acts as the PLC runtime, receiving uploaded ladder logic and executing it using its digital input/output pins.
3. **Hardware I/O Circuit** – includes LED output, START and STOP push buttons, resistors, jumper wires, and breadboard connections.

The setup supports two tasks:

- **LED blinking circuit** using a timer coil.
- **Start–Stop motor control-style circuit** using latching logic with push buttons and one LED.

4.0 METHODOLOGY

4.1 Software Preparation

1. Download and install **OpenPLC Editor**.
2. Launch the software and create a new project folder.
3. Create a new Program Organization Unit (POU) and set the programming language to **Ladder Diagram (LD)**.

4.2 Ladder Diagram Creation

1. Create project variables (Class: Local, Type: BOOL).
2. Construct the left and right power rails.
3. Add required elements: Normally open/closed contacts, coils, and timer blocks (if needed).
4. Build the blinking LED ladder diagram according to the provided figure.
5. Compile the program in OpenPLC to check for errors and run a simulation.

4.3 Physical Address Mapping

1. Identify the output pin used on the Arduino.
2. Refer to OpenPLC documentation to determine the correct digital output address (e.g., Arduino Mega pin 14 = %QX0.0).
3. Assign the address to the output variable in the ladder diagram.

4.4 Uploading Ladder Logic to the Arduino

1. Connect the Arduino board to the PC using USB.
2. Note the COM port number.
3. Press the “Transfer” icon in OpenPLC.
4. Select the correct Arduino board model.
5. Type the correct COM port and upload the program.

4.5 Hardware Setup

For the Start–Stop control circuit:

- Push button for START
- Push button for STOP
- LED output
- Resistors (pull-down or pull-up as needed)
- Breadboard and jumper wires

Connections were made as shown in the experiment figures to ensure proper signal inputs and outputs.

4.6 Testing and Verification

1. Power the Arduino and run the uploaded OpenPLC program.
2. Observe LED blinking for the first exercise.
3. For the Start–Stop circuit:
 - Press START button → LED turns ON and latches.
 - Press STOP button → LED turns OFF.
4. Record results and troubleshoot any incorrect behavior.

5.0 DATA COLLECTION

Data were collected by observing and recording the behavior of both circuits after uploading the ladder logic program. Table 5.1 summarizes the types of data gathered for each experiment.

Experiment	Data Collected	Description
LED Blinking Circuit	LED ON/OFF timing	Comparison of measured interval vs TON setting
	Timing drift	Slight deviations due to PLC scan cycle
Start–Stop Control Circuit	Input state changes	START and STOP button behavior
	Output LED response	Latching/unlatching behavior
	State transition log	Recorded in table form during testing

Table 5.1: Summary of Collected Data

6.0 DATA ANALYSIS

6.1 LED Blinking Circuit (Timer-Based Analysis)

The measured time interval closely matched the TON value, with slight delays caused by:

- Microcontroller cycle latency
- Serial communication overhead
- Internal PLC scan cycle delay

The PLC scan cycle introduces a consistent overhead of approximately 3–10 ms depending on the TON duration. This is expected in PLC-based timing systems where program scan time adds to total delay.

6.2 Start–Stop Control Circuit (Latching Logic Analysis)

Logic Behavior

The ladder diagram consisted of:

- Start button (NO contact)
- Stop button (NC contact)
- Latching coil (self-hold)
- Output coil controlling LED or actuator

Test step	Start Button	Stop Button	LED Output	Result
Initial	Not Pressed	Not Pressed	OFF	Correct
Press Start	LOW Pulse	Not Pressed	ON	Latch Engaged
Hold Start	LOW	Not Pressed	ON	No Change
Release Start	HIGH	Not Pressed	ON	Latch Holds
Press Stop	Not Pressed	LOW	OFF	Latch Breaks

Table 6.1 : Observed Output States During Testing

The latching behavior matched standard PLC industrial start–stop motor control logic. The Arduino board successfully replicated PLC behavior due to the OpenPLC runtime environment. The logic operated consistently in both simulation and hardware.

7.0 RESULTS

The results confirm that both ladder logic programs performed correctly when executed through the Arduino-based OpenPLC runtime. Tables 7.1 and 7.2 present the outcomes for each experiment.

Parameter	Expected Behavior	Observed Behavior
TON Timer Interval	LED toggles at programmed timing	LED toggled at nearly the same interval
Timing Accuracy	Minimal deviation	Slight delay due to scan cycle and microcontroller latency
Program Execution	Smooth and continuous	Operated consistently without error

Table 7.1: LED Blinking Circuit Results

Condition	Expected Output	Observed Output	Outcome
Press START	LED turns ON and latches	LED latched ON correctly	✓
Release START	LED remains ON	LED remained ON	✓
Press STOP	LED turns OFF and latch breaks	LED turned OFF	✓
Multiple Cycles	Repeated stable operation	No mis-triggering or instability	✓

Table 7.2: Start–Stop Circuit Results

8.0 DISCUSSIONS

The experiment demonstrated the fundamentals of PLC logic development and deployment in a cost-effective manner using OpenPLC and Arduino hardware. Creating the ladder diagram helped students better understand PLC logic elements like contacts, coils, latching mechanisms, and timers. Simulation within OpenPLC provided an effective environment for testing logic prior to uploading it to hardware.

Physical addressing was critical in ensuring that PLC variables corresponded correctly to Arduino digital I/O pins. Misaligned addresses would cause the program to behave incorrectly, highlighting the importance of understanding I/O mapping in automation systems.

Uploading the program via Ethernet/IP enabled the Arduino to function as a PLC runtime device. The successful blinking LED program validated the ladder diagram's basic functionality. The behavior of the Start-Stop circuit matched industrial motor-control logic, with the START button initiating a latch and the STOP button breaking the circuit.

Overall, the experiment demonstrated that OpenPLC is a useful educational tool for learning PLC concepts that does not require industrial PLC units. The setup combines theoretical knowledge with practical implementation, improving students' understanding of automation system integration.

9.0 CONCLUSION

This experiment successfully demonstrated the interfacing of a PLC-based ladder logic program with an Arduino microcontroller using the OpenPLC platform. The use of Ethernet/IP-style programming and ladder diagrams allowed the Arduino to behave as a small-scale PLC, enabling real industrial control logic to be executed on low-cost hardware.

1. The **timer-based LED blinking** worked correctly, and modifying the TON value directly affected the LED's blinking frequency with minimal deviation.
2. The **Start–Stop Control Circuit** functioned as expected, demonstrating proper latching and unlatching behavior widely used in industrial motor control circuits.
3. The OpenPLC simulation results closely matched the real hardware behavior, showing that the platform is highly effective for educational PLC training and prototyping.
4. Interfacing between PC → OpenPLC → Arduino was stable, proving that microcontrollers can serve as PLC hardware substitutes for entry-level automation.

Overall, the experiment reinforced foundational concepts in industrial automation, PLC programming, timing control, and microcontroller interfacing.

10.0 RECOMMENDATION

1. Use Debouncing on Pushbuttons

Mechanical switches generate noise during actuation. Adding software or hardware debouncing would improve reliability and prevent false start/stop triggers.

2. Optimize Scan Time for Timing Accuracy

The minor timing delays observed are due to PLC scan cycles. Reducing unnecessary rungs or simplifying logic can minimize execution time and improve accuracy.

3. Include Safety Interlocks

For real industrial applications, add components such as:

- Emergency stop
- Overload protection virtual contacts
- Logical fail-safe states

4. Expand to Multiple Outputs

Future experiments can include controlling:

- Motors (via relays or motor drivers)
- Buzzer alarms
- Solenoids and pneumatic valves

5. Add Analog Inputs

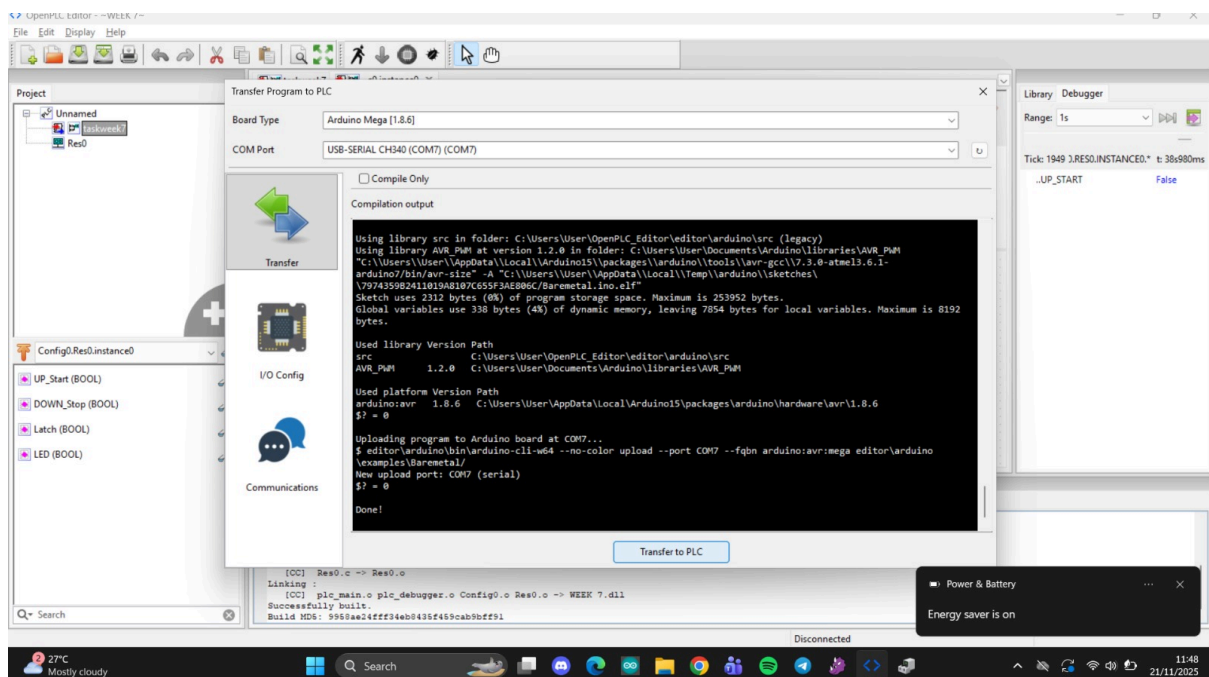
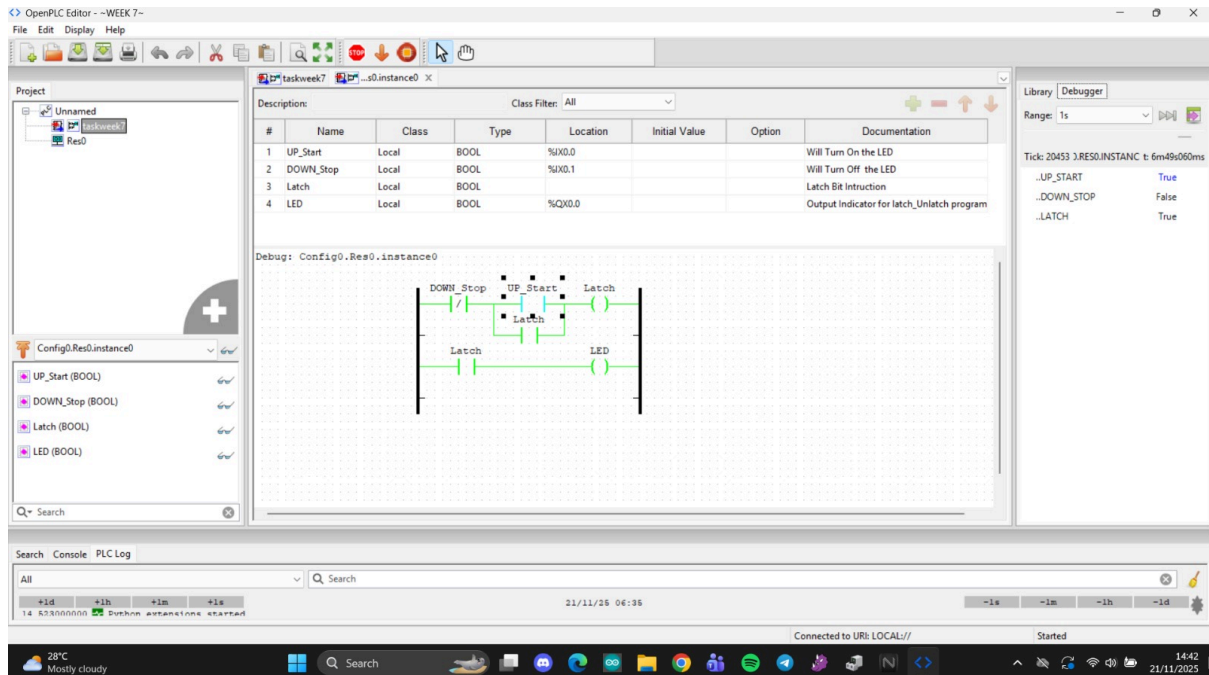
Future exercises may incorporate analog sensors (potentiometer, temperature sensor) to explore scaling, filtering, and signal conditioning.

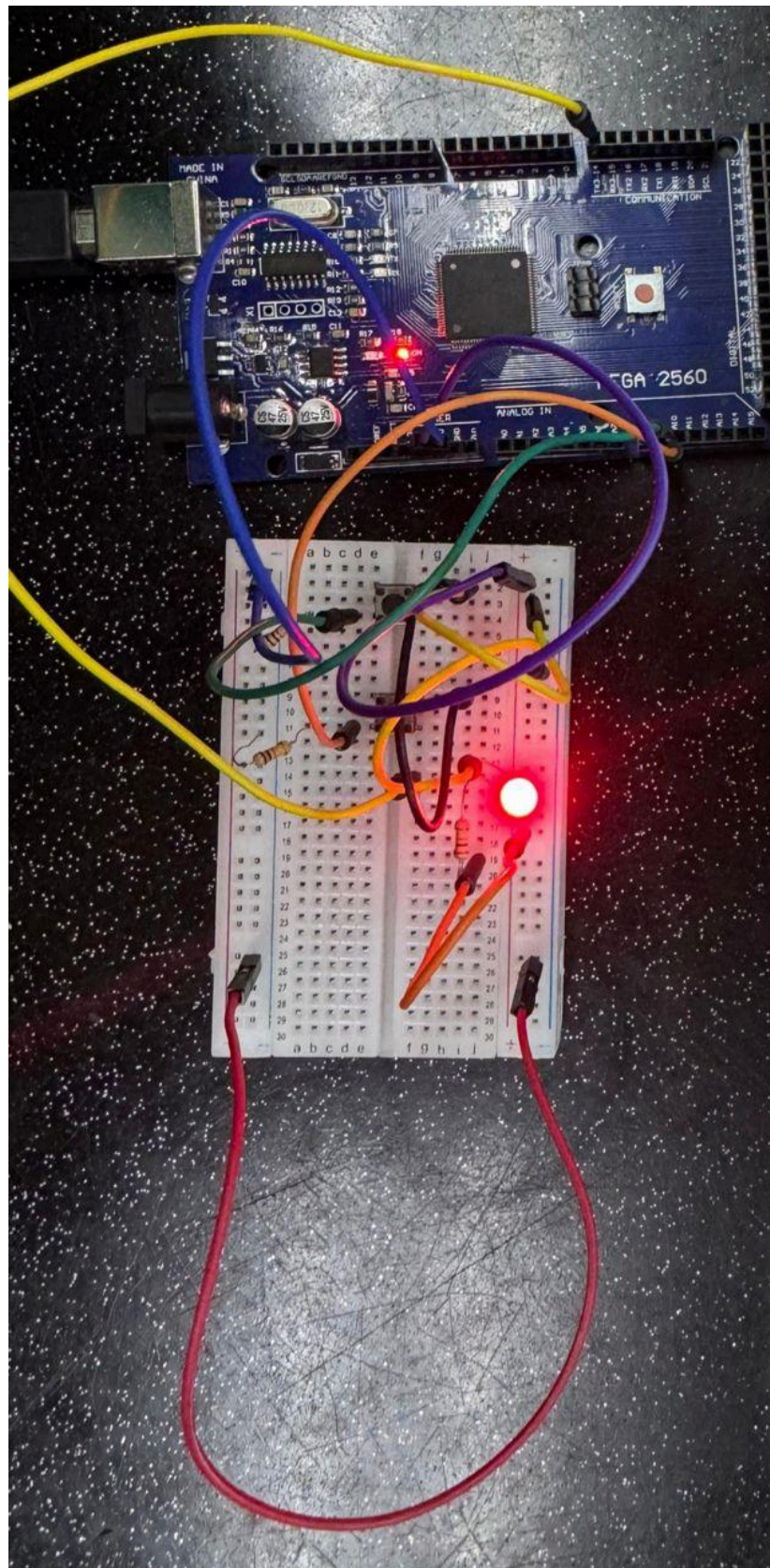
REFERENCES

Wilcher, D. (2022, August 30). *PLC ladder logic on an Arduino: Introduction to OpenPLC*. Control. Retrieved from <https://control.com/technical-articles/plc-ladder-logic-on-an-arduino-introduction-to-openplc/>

APPENDICES

Appendix A





ACKNOWLEDGEMENT

We would like to sincerely thank the instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal Abidin & Dr. Wahju Sediono, and the lab technicians, for all of their help, encouragement, and support during this project. Their knowledge and perceptions have greatly influenced the course of this work. Additionally, we would like to express our gratitude to our peers for their support and cooperation, both of which were crucial to the accomplishment of this project.

STUDENTS DECLARATION

We hereby declare that, except for the places where it is acknowledged, all of the work presented in this report is entirely ours. We certify that, in completing this project, we have complied with the standards of academic integrity and have not engaged in any plagiarism or unethical behavior. Every information and support source used in this work has been appropriately referenced and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has **not been done by only one individual, and all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read and understand** the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for **marking**, and this **final printed report** has been **verified by us**.

Signature: *haikalzulhilmi*

Name: Muhammad Haikal Zulhilmi Bin Fairof
Matric Number: 2313025

Read [/]
Understand [/]
Agree [/]

Signature: *faizizwan*

Name: Muhammd Faiz Izwan Bin Nor Effendi
Matric Number: 2313509

Read [/]
Understand [/]
Agree [/]

Signature: *Ainul Jaariah*

Name: Ainul Jaariah Binti Aliudin
Matric Number: 2312664

Read [/]
Understand [/]
Agree [/]